

BÀI GIẢNG 3 TIẾT

Lập trình hướng đối tượng với Java

class
→ object

Từ tư duy mô hình hóa đến chương trình để mở rộng

Mở đầu

©IT_FIT_2026

1

1

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VỚI JAVA

Chương trình lớn không khó vì cú pháp — mà vì độ phức tạp

100 dòng

Có thể giữ toàn bộ luồng xử lý trong đầu

10.000 dòng

Dữ liệu và thao tác đan xen; một thay đổi kéo theo nhiều lỗi

100.000 dòng

Cần chia trách nhiệm, tạo ranh giới và hợp đồng rõ ràng

OOP là một cách tổ chức độ phức tạp — không chỉ là tập hợp từ khóa Java.

Mở đầu

©IT_FIT_2026

2

2

Vì sao sinh viên cần học OOP với Java?

1

Mô hình hóa bài toán

Biểu diễn sinh viên, tài khoản, cảm biến... thành các đối tượng gần với thế giới thực.

2

Kiểm soát thay đổi

Đóng gói dữ liệu và giới hạn tác động khi sửa một thành phần.

3

Tái sử dụng và mở rộng

Tạo loại mới từ thiết kế có sẵn; thay thế hành vi qua đa hình.

4

Nền tảng nghề nghiệp

Java API, Spring, Android và nhiều framework được thiết kế quanh lớp, interface và đối tượng.

Java giúp nhìn rõ OOP nhờ kiểu dữ liệu chặt chẽ, class/interface tường minh và hệ sinh thái lớn.

Mở đầu

©IT_FIT_2026

3

3

TIẾT 1 • 45 PHÚT

Từ dữ liệu rời rạc đến lớp và đối tượng

Khái niệm • Class • Object • Constructor • this

Tiết 1

©IT_FIT_2026

5

5

Cùng một bài toán, hai cách tổ chức chương trình

Tư duy thủ tục

JAVA

```
String tenSV = "An";
double diemTB = 8.5;

void hienThi(String tenSV, double diemTB) {
    System.out.println(tenSV + ": " +
        diemTB);
}
```

Dữ liệu và hàm tách rời; khó bảo đảm chúng luôn được dùng đúng cùng nhau.

Tư duy đối tượng

JAVA

```
class SinhVien {
    String tenSV;
    double diemTB;

    void hienThi() {
        System.out.println(tenSV + ": " +
            diemTB);
    }
}
```

Dữ liệu và hành vi liên quan được đặt trong cùng một đơn vị có ý nghĩa.

Class là bản thiết kế; object là thực thể được tạo từ bản thiết kế

CLASS: SinhVien

- Thuộc tính: maSV, hoTen, diemTB
- Hành vi: datHocBong(), hienThiThongTin()
- Mô tả cấu trúc chung

Object sv1

SV001 • Nguyễn An • 8.5

Object sv2

SV002 • Trần Bình • 7.2

Một class có thể tạo nhiều object; mỗi object có trạng thái riêng.

Một lớp Java kết hợp trạng thái và hành vi

JAVA

```
public class SinhVien {
    String maSV;           // thuộc tính
    String hoTen;
    double diemTB;

    void hienThiThongTin() { // phương thức
        System.out.println(maSV + " - " + hoTen);
    }

    boolean datHocBong() {
        return diemTB >= 8.0;
    }
}
```

A

Thuộc tính

Biến mô tả trạng thái của đối tượng.

B

Phương thức

Hành vi mà đối tượng có thể thực hiện.

C

Trách nhiệm

Lớp nên chứa dữ liệu và hành vi có liên quan.

Tạo object bằng new, truy cập thuộc tính/phương thức bằng dấu chấm

JAVA

```
public class Main {
    public static void main(String[] args) {
        SinhVien sv1 = new SinhVien();
        sv1.maSV = "SV001";
        sv1.hoTen = "Nguyễn An";
        sv1.diemTB = 8.5;

        sv1.hienThiThongTin();
        System.out.println(sv1.datHocBong());
    }
}
```

Đọc câu lệnh

- SinhVien: kiểu tham chiếu
- sv1: biến tham chiếu
- new SinhVien(): tạo object
- sv1.diemTB: truy cập thành viên

Kết quả

SV001 - Nguyễn An
true

Constructor tạo trạng thái hợp lệ ngay khi object xuất hiện

JAVA

```
public class SinhVien {
    String maSV;
    String hoTen;
    double diemTB;

    public SinhVien(String maSV, String hoTen,
                    double diemTB) {
        this.maSV = maSV;
        this.hoTen = hoTen;
        this.diemTB = diemTB;
    }
}

SinhVien sv = new SinhVien("SV001", "Nguyễn An", 8.5);
```

Ba ý cần nhớ

- Tên constructor trùng tên class
- Không khai báo kiểu trả về
- this.maSV là thuộc tính; maSV là tham số

Constructor giúp tránh object thiếu mã sinh viên hoặc chưa có tên.

Hoạt động 7 phút: xây dựng lớp SanPham

Yêu cầu

- Thuộc tính: maSP, tenSP, donGia, soLuong
- Constructor đầy đủ tham số
- tinhThanhTien() = donGia × soLuong
- hienThiThongTin() in thông tin và thành tiền

Kiểm thử tối thiểu

TEST

```
SanPham sp = new SanPham(
    "SP01", "Bàn phím", 350000, 2
);
sp.hienThi();
// Thành tiền mong đợi: 700000
```

Cặp đôi: một người viết class, một người viết Main; đổi vai sau 4 phút.

Kiểm tra nhanh cuối tiết 1

- 1 Class và object khác nhau như thế nào?
- 2 Constructor được gọi vào thời điểm nào?
- 3 this.maSV và maSV có thể chỉ hai thứ khác nhau khi nào?

Mục tiêu: giải thích bằng lời của chính mình, không đọc định nghĩa.

Bốn đặc trưng làm nên sức mạnh của OOP

Đóng gói • Kế thừa • Đa hình • Trừu tượng

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VỚI JAVA

Đóng gói bảo vệ tính hợp lệ của trạng thái

- Dữ liệu không cho truy cập trực tiếp
- Chỉ thao tác thông qua getter/setter
- Kiểm soát tính hợp lệ của dữ liệu

JAVA

```
public class TaiKhoan {
    private double soDu;

    public double getSoDu() {
        return soDu;
    }

    public void napTien(double soTien) {
        if (soTien <= 0) {
            throw new IllegalArgumentException();
        }
        soDu += soTien;
    }
}
```

Nếu soDu là public

NGUY CƠ

```
taiKhoan.soDu = -1_000_000;
```

Khi soDu là private

- Mọi thay đổi đi qua phương thức
- Có nơi duy nhất để kiểm tra dữ liệu
- Không đồng nghĩa phải tạo setter cho mọi thuộc tính

Tiết 2 • Đóng gói

©IT_FIT_2026

14

14

14

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VỚI JAVA

Kế thừa mô tả quan hệ “là một” và tái sử dụng có kiểm soát

- Lớp con **kế thừa lại code** của lớp cha
- Tránh viết lại code giống nhau → tái sử dụng code

JAVA

```
class NhanVien {
    protected String hoTen;

    public double tinhLuong() {
        return 0;
    }
}

class NhanVienToanThoiGian extends NhanVien {
    private double luongThang;

    @Override
    public double tinhLuong() {
        return luongThang;
    }
}
```

Kiểm tra trước khi extends

- B có thật sự là một A?
- B có thể thay thế A không?
- Hành vi chung có ổn định không?

**NhanVienToanThoiGian là một
NhanVien ✓**

Tiết 2 • Kế thừa

©IT_FIT_2026

15

15

15

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VỚI JAVA

Đa hình cho phép một lời gọi tạo ra nhiều hành vi

- Cùng 1 lời gọi phương thức
- Nhưng hành vi khác nhau
- Xây ra khi **override** phương thức

JAVA

```

Nhanvien nv = new Nhanvien();
NhanVien nv1 = new NhanVienToanThoiGian();
NhanVien nv2 = new NhanVienBanThoiGian();

List<NhanVien> ds = List.of(nv1, nv2);

for (NhanVien nv : ds) {
    System.out.println(nv.tinhLuong());
}

```

Kiểu khai báo

NhanVien

Đối tượng thực tế

ToanThoiGian /
BanThoiGian

Phương thức chạy

Phiên bản override của object

Thêm loại nhân viên mới mà không sửa vòng lặp.

Tiết 2 • Đa hình

©IT_FIT_2026

16

16

16

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VỚI JAVA

Trừu tượng giữ lại điều quan trọng và che đi chi tiết triển khai

- Không quan tâm làm thế nào
- Chỉ quan tâm làm cái gì
- Dùng **abstract class** hoặc **interface**

JAVA

```

abstract class Hinh {
    private String mauSac;

    public abstract double tinhDienTich();

    public void hienThiMau() {
        System.out.println(mauSac);
    }
}

class HinhTron extends Hinh {
    private double banKinh;

    @Override
    public double tinhDienTich() {
        return Math.PI * banKinh * banKinh;
    }
}

```

abstract class có thể

- Có thuộc tính và constructor
- Có phương thức thường
- Có phương thức abstract
- Không tạo object trực tiếp

Hinh quy định “phải tính được diện tích”; mỗi hình tự quyết định cách tính.

Tiết 2 • Trừu tượng

©IT_FIT_2026

17

17

17

Interface định nghĩa năng lực chung giữa các lớp khác nhau

JAVA

```
interface CoTheBatTat {
    void bat();
    void tat();
}

class DenThongMinh implements CoTheBatTat {
    public void bat() {
        System.out.println("Đèn bật");
    }

    public void tat() {
        System.out.println("Đèn tắt");
    }
}
```

Dùng interface khi

- Các lớp cần chung một phương thức
- Không cần chung trạng thái/lớp cha
- Muốn giảm phụ thuộc vào triển khai cụ thể

Một lớp Java chỉ extends một class, nhưng có thể implements nhiều interface.

Bốn đặc trưng giải quyết bốn câu hỏi thiết kế khác nhau

Đóng gói

Ai được phép đổi trạng thái?

private +
method

Kế thừa

Loại nào là phiên bản chuyên biệt?

extends

Đa hình

Cùng lời gọi, hành vi nào chạy?

override + kiểu
cha

Trừu tượng

Điều gì cần biết; điều gì có thể ẩn?

abstract /
interface

Thiết kế tốt thường kết hợp cả bốn — nhưng chỉ dùng khi chúng làm trách nhiệm rõ hơn.

Những ngộ nhận thường làm thiết kế OOP trở nên rườm rà

Ngộ nhận

- Mọi thuộc tính đều cần getter và setter
- Càng nhiều kế thừa càng tái sử dụng tốt
- Một class chứa càng nhiều chức năng càng tiện
- OOP luôn tốt hơn lập trình thủ tục

Cách sửa

- Chỉ mở hành vi mà đối tượng thật sự cần
- Ưu tiên quan hệ đúng nghĩa và composition khi phù hợp
- Mỗi class có trách nhiệm tập trung
- Chọn mô hình theo độ phức tạp của bài toán

Dấu hiệu tốt: có thể mô tả trách nhiệm của class bằng một câu ngắn.

Hoạt động 6 phút: chọn công cụ OOP phù hợp

- | | | |
|----------|--|-----|
| A | Không cho số dư tài khoản âm | → ? |
| B | Hình tròn và hình chữ nhật cùng tính diện tích | → ? |
| C | Đèn và quạt đều có thể bật/tắt | → ? |
| D | Nhân viên toàn thời gian là một nhân viên | → ? |

Che cột đáp án khi trình chiếu; yêu cầu nhóm giải thích "vì sao".

Câu hỏi kiểm tra cuối tiết 2

Cho đoạn mã:

```
JAVA
CoTheBatTat x = new DenThongMinh();
x.bat();
```

Câu 1

Kiểu khai báo của x là gì?

Câu 2

Object thực tế thuộc lớp nào?

Câu 3

Đặc trưng OOP nào xuất hiện?

Trao đổi theo cặp trong 30 giây trước khi trả lời.